Assignment #3: Data and Query Design

Learning Assistant — Team 08

Platform Development Course · Digital Campus Platform · Track 2 (Advanced Edge)
Stack: Cloudflare Workers · Hono · TypeScript · Cloudflare D1

1. Introduction

Learning Assistant is a course question-answering service built as part of a larger university Digital Campus Platform. It lets a student ask a course-scoped question and receive an answer that is grounded **only** in materials an instructor or teaching assistant has explicitly approved for that course.

Two properties define the product:

- **Every substantive answer carries citations.** An answer is either backed by one or more approved knowledge sources (with excerpts a student can verify), or the system honestly reports that it found nothing and points the student to their instructor — it never invents an answer.
- **It is one service among many.** In the Digital Campus Platform, each team owns one focused service with its own data boundary, integrating with the rest of the platform through REST APIs and versioned webhook events rather than shared databases.

This assignment covers the data and query design for that service: the relational schema, why the chosen database fits, and the REST API that reads and writes it.

2. Database Choice

Cloudflare D1 (a managed, serverless SQLite database) was chosen for Learning Assistant.

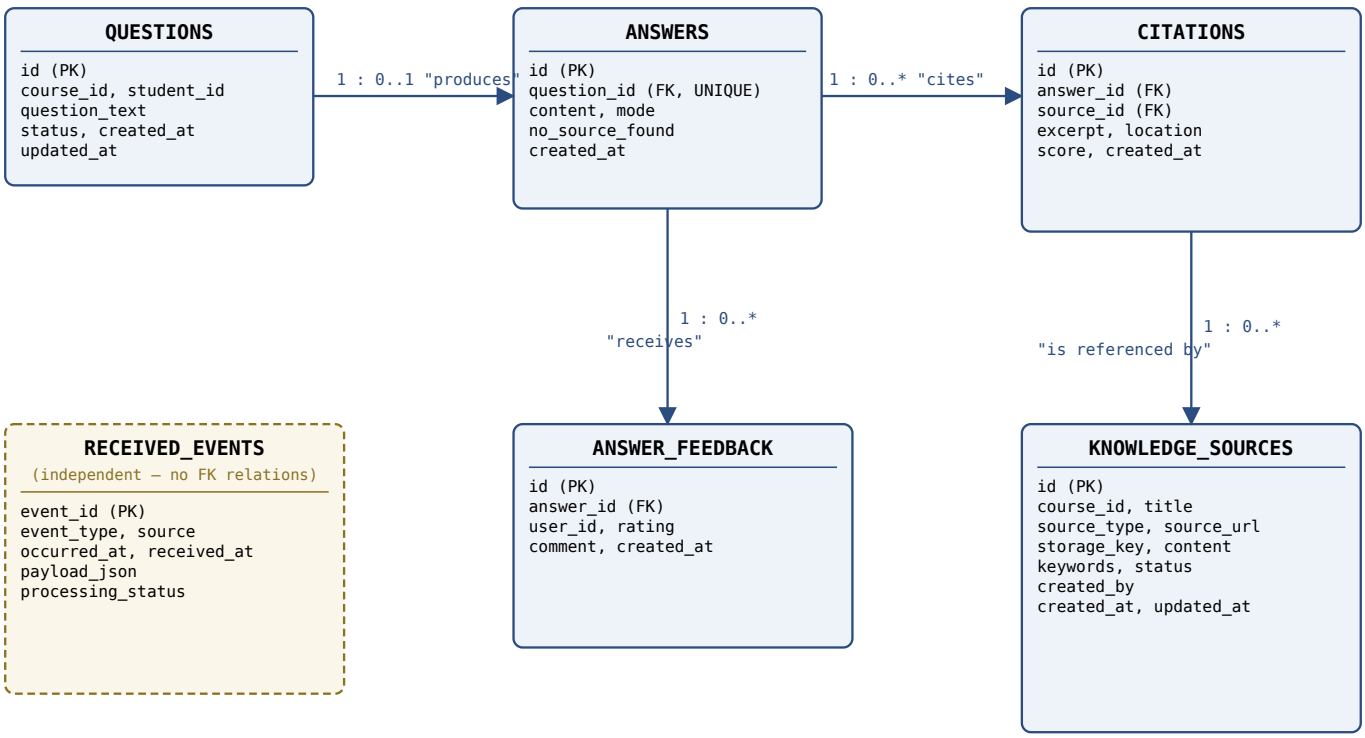
Reason	Why it matters here
SQLite-compatible relational database	The domain is inherently relational: a course has sources; a question has (at most) one answer; an answer has citations and feedback. Foreign keys and UNIQUE/CHECK constraints express these rules directly in the schema.
Native Cloudflare Worker integration	The API is a Hono app on Cloudflare Workers. D1 is bound directly into the Worker (`env.learning_assistant_db`) with no separate network hop or driver.
Simple migrations, local and remote	`wrangler d1 migrations apply ... --local / --remote` runs the same numbered SQL migration against a local simulated database or the real remote one.
Parameterized SQL, indexes, FKs, CHECK constraints	Standard SQLite: prepared statements with bound parameters, indexes, foreign keys with ON DELETE behavior, column-level CHECK constraints — everything this schema needed.
Cost / operational simplicity for a student MVP	Workable free tier, no server to provision or patch, scales to zero between requests — appropriate for a course project.

A realistic limitation: D1 is suitable for this relational MVP, but it does not provide native vector search for future semantic retrieval — that would need an additional component if added later.

3. Data Ownership and Boundaries

- Learning Assistant owns **its own** D1 database and schema exclusively. No other Digital Campus Platform service reads or writes these tables directly, and this service does not reach into any other service's data store either.
- `course_id`, `student_id`, `user_id`, and `created_by` are **external identifiers** supplied by other platform services (Course Catalog, Student Identity), stored here only as opaque references.
- Cross-service integration happens through **REST APIs and signed webhook events**, never direct database access across service boundaries.
- `received_events` exists to support this: an idempotency ledger for **future** inbound webhook processing (e.g. `assignment.published`). Its primary key is the event id, so redelivery is safe to detect and skip.

4. Entity Relationship Diagram



Mermaid ER diagram source (also stored standalone at `docs/diagrams/learning-assistant-er-diagram.mmd` and inline in the Markdown report):

```
erDiagram
    QUESTIONS ||--o| ANSWERS : "produces"
    ANSWERS ||--o{ CITATIONS : "cites"
    KNOWLEDGE_SOURCES ||--o{ CITATIONS : "is referenced by"
    ANSWERS ||--o{ ANSWER_FEEDBACK : "receives"
    RECEIVED_EVENTS { }  (independent, no relationships)
```

Reading the diagram: a **question** produces at most **one answer** (enforced by `answers.question_id` being both a foreign key and UNIQUE); an **answer** cites **zero or more citations** (zero exactly when `no_source_found = 1`); a **knowledge source** can be referenced by **zero or more citations**; an **answer** can receive **zero or more feedback** records; **received_events** is independent, with no foreign-key relationship to any other table.

5. Relational Schema

The full, authoritative schema lives in `migrations/0001_initial_learning_assistant_schema.sql`.

knowledge_sources

Instructor/TA-approved learning material for one course — the only thing the fallback search is ever allowed to draw an answer from.

Column	Type	Constraints	Purpose
id	TEXT	PRIMARY KEY	Source identifier (UUID)
course_id	TEXT	NOT NULL	Owning course (external identifier)
title	TEXT	NOT NULL	Display title; also searched by the fallback matcher
source_type	TEXT	NOT NULL	e.g. document, link, text
source_url	TEXT	nullable	Set when the source is a link
storage_key	TEXT	nullable	Set when the original file is stored elsewhere
content	TEXT	nullable	Extracted/plain text used for matching
keywords	TEXT	nullable	Comma-separated keyword list; also searched
status	TEXT	NOT NULL, DEFAULT 'approved', CHECK IN (draft, approved, archived)	Lifecycle state
created_by	TEXT	NOT NULL	Instructor/TA identifier (external)
created_at / updated_at	TEXT	NOT NULL	UTC ISO-8601 timestamps

Soft archive: `DELETE /v1/sources/:id` never removes this row. It sets `status = 'archived'` and refreshes `updated_at`. An archived source is immediately excluded from future retrieval, while any citation pointing to it is preserved as history.

questions

Column	Type	Constraints	Purpose
id	TEXT	PRIMARY KEY	Question identifier (UUID)
course_id	TEXT	NOT NULL	Course the question was asked in
student_id	TEXT	NOT NULL	Asking student (from identity headers, never the body)
question_text	TEXT	NOT NULL	The question itself
status	TEXT	NOT NULL, DEFAULT 'pending', CHECK IN (pending, answered, no_source_found, failed)	Lifecycle state
created_at / updated_at	TEXT	NOT NULL	UTC ISO-8601 timestamps

answers

Column	Type	Constraints	Purpose
id	TEXT	PRIMARY KEY	Answer identifier (UUID)
question_id	TEXT	NOT NULL, UNIQUE, FK → questions(id) ON DELETE CASCADE	One answer per question
content	TEXT	NOT NULL	The templated answer text
mode	TEXT	NOT NULL, CHECK IN (ai, fallback)	How the answer was produced (currently always fallback)
no_source_found	INTEGER	NOT NULL, DEFAULT 0, CHECK IN (0,1)	1 when nothing approved matched
created_at	TEXT	NOT NULL	UTC ISO-8601 timestamp

citations

Column	Type	Constraints	Purpose
id	TEXT	PRIMARY KEY	Citation identifier (UUID)
answer_id	TEXT	NOT NULL, FK → answers(id) ON DELETE CASCADE	Owning answer
source_id	TEXT	NOT NULL, FK → knowledge_sources(id) ON DELETE RESTRICT	Cited source
excerpt	TEXT	NOT NULL	Short verifiable text from the source
location	TEXT	nullable	Reserved for a page/section reference
score	REAL	nullable	Fallback-search match score
created_at	TEXT	NOT NULL	UTC ISO-8601 timestamp

The two foreign keys use different `ON DELETE` behavior deliberately: CASCADE on `answer_id`, RESTRICT on `source_id` — a source that is still cited cannot disappear; it must be archived instead.

answer_feedback

Column	Type	Constraints	Purpose
id	TEXT	PRIMARY KEY	Feedback identifier (UUID)
answer_id	TEXT	NOT NULL, FK → answers(id) ON DELETE CASCADE	Rated answer
user_id	TEXT	NOT NULL	Rating user (external identifier)
rating	INTEGER	NOT NULL, CHECK BETWEEN 1 AND 5	1–5 rating
comment	TEXT	nullable	Optional free-text comment
created_at	TEXT	NOT NULL	UTC ISO-8601 timestamp

received_events

Column	Type	Constraints	Purpose
event_id	TEXT	PRIMARY KEY	The idempotency key
event_type	TEXT	NOT NULL	e.g. assignment.published
source	TEXT	NOT NULL	Publishing service
occurred_at	TEXT	NOT NULL	When the source event happened
received_at	TEXT	NOT NULL	When this service received it
payload_json	TEXT	NOT NULL	Raw event payload
processing_status	TEXT	NOT NULL, DEFAULT 'received', CHECK IN (received, processed, failed)	Processing lifecycle

Indexes

Index	Table (columns)	Why
idx_knowledge_sources_course_status	knowledge_sources(course_id, status)	Listing always filters by course and status = approved
idx_knowledge_sources_course_title	knowledge_sources(course_id, title)	Listing is ordered by title within a course
idx_questions_student_created	questions(student_id, created_at)	A student's own question history
idx_questions_course_created	questions(course_id, created_at)	A course's question history
idx_citations_answer_id	citations(answer_id)	Fetching all citations for one answer
idx_citations_source_id	citations(source_id)	Finding citations referencing a given source
idx_answer_feedback_answer_id	answer_feedback(answer_id)	Fetching feedback for an answer
idx_received_events_type_received	received_events(event_type, received_at)	Future: scanning recent events by type

6. Database Migration and Deployment

Migration file: `migrations/0001_initial_learning_assistant_schema.sql` — a single, additive migration creating all six tables and eight indexes.

Local migration verification (actually run)

```
rm -rf .wrangler/state/v3/d1
npx wrangler d1 migrations apply learning-assistant-db --local
```

Outcome: **15 SQL statements executed successfully** (6 CREATE TABLE + 8 CREATE INDEX + 1 internal migration-tracking insert). Table/index presence and the two documented column defaults (`questions.status = 'pending'` , `received_events.processing_status = 'received'`) were confirmed by inserting rows that omit those columns and reading the value back. Full detail in `docs/assignment-3-evidence/local-schema-verification.txt` .

Remote migration procedure

```
npx wrangler login
npx wrangler d1 migrations apply learning-assistant-db --remote
npx wrangler d1 execute learning-assistant-db --remote --command "SELECT name, type FROM
sqlite_master WHERE type IN ('table', 'index') ORDER BY type, name;"
```

> **Remote deployment status: Confirmed.** With team approval, the above commands were run against the real `learning-assistant-db` database. `wrangler` reported “No migrations to apply!” — the remote database already had migration `0001_initial_learning_assistant_schema.sql` recorded as applied (most likely

> from an earlier deploy by a teammate on this shared repository). The remote schema was then independently verified: all **6 application tables** and all **8 explicit indexes** are present on the remote database, exactly matching the local schema. No destructive action occurred and no data was changed (`changed_db: false` , `rows_written: 0`). Full detail in `docs/assignment-3-evidence/remote-migration-verification.txt` .

7. API and CRUD Design

Method	Path	Role	Purpose
GET	/v1/health	none	Liveness check
POST	/v1/sources	instructor	Create a knowledge source
PATCH	/v1/sources/{id}	instructor	Update a knowledge source
DELETE	/v1/sources/{id}	instructor	Archive (soft delete) a knowledge source
GET	/v1/courses/{courseId}/sources	student, instructor	List a course's approved sources
POST	/v1/questions	student	Ask a question; get a fallback answer + citations
GET	/v1/questions/{id}	owning student, instructor	Read a question with its answer and citations
POST	/v1/answers/feedback	student, instructor	Rate an answer (1–5)

CRUD mapped onto the sources resource

Operation	Endpoint	Notes
Create	POST /v1/sources	Instructor only; created_by from identity header, never the body
Read	GET /v1/courses/{courseId}/sources	Returns only status = approved sources for that course
Update	PATCH /v1/sources/{id}	Instructor only; updates only supplied fields plus updated_at
Archive (in place of hard Delete)	DELETE /v1/sources/{id}	Instructor only; see below

DELETE is intentionally a soft archive, not a hard delete: the row is never physically removed; `status` becomes `archived` and `updated_at` refreshes; the source immediately disappears from future listings; any citation already pointing at it is preserved; the operation is idempotent.

8. API Demonstration

All examples were run locally against `npx wrangler dev --local` using the fictional course `SE101` and fictional users `instructor-demo`, `student-demo-a`, `student-demo-b`. Full evidence in `docs/assignment-3-evidence/`.

Temporary development headers. Every `/v1` route except `/v1/health` requires `x-user-id` and `x-user-role` (`student` or `instructor`). These are **scaffolding only** — no signature or token verification — intended to be replaced by real Student Identity token validation later.

Health

```
curl -s http://127.0.0.1:8787/v1/health
```

```
{"status":"ok","service":"learning-assistant"}
```

Create a source

```
curl -s -X POST http://127.0.0.1:8787/v1/sources \
  -H 'x-user-id: instructor-demo' -H 'x-user-role: instructor' \
  -H 'Content-Type: application/json' \
  -d '{
    "courseId": "SE101",
    "title": "Week 3 REST API Lecture Notes",
    "sourceType": "document",
    "sourceUrl": "https://example.edu/se101/week-3-rest",
    "content": "REST is an architectural style for networked applications...",
    "keywords": ["REST", "API", "HTTP", "GET", "POST"],
    "status": "approved"
  }'
```

```
HTTP 201
{ "data": { "id": "<source-id>", "courseId": "SE101", "title": "Week 3 REST API Lecture Notes",
  "status": "approved", "createdBy": "instructor-demo", "...": "..." } }
```

List approved sources (a draft source stays hidden)

```
curl -s http://127.0.0.1:8787/v1/courses/SE101/sources \
  -H 'x-user-id: student-demo-a' -H 'x-user-role: student'
```

```
HTTP 200
{ "data": { "items": [ { "id": "<source-id>", "status": "approved", "...": "..." } ], "count": 1
} }
```

Update a source

```
curl -s -X PATCH http://127.0.0.1:8787/v1/sources/<source-id> \
  -H 'x-user-id: instructor-demo' -H 'x-user-role: instructor' \
  -H 'Content-Type: application/json' \
  -d '{ "title": "Week 3 REST API Lecture Notes (Revised)", "keywords":
["REST","API","HTTP","GET","POST","PATCH","DELETE"] }'
```

```
HTTP 200
{ "data": { "id": "<source-id>", "title": "Week 3 REST API Lecture Notes (Revised)",
  "keywords": "REST, API, HTTP, GET, POST, PATCH, DELETE", "status": "approved", "...": "..." }
}
```

Archive a source

```
curl -s -X DELETE http://127.0.0.1:8787/v1/sources/<source-id> \
  -H 'x-user-id: instructor-demo' -H 'x-user-role: instructor'
```

```
HTTP 200
{ "data": { "id": "<source-id>", "status": "archived", "...": "..." } }
```

Listing again afterward returns `{ "data": { "items": [], "count": 0 } }`. Repeating the same DELETE returns `200` again (idempotent).

Matching question → fallback answer with a citation

```
curl -s -X POST http://127.0.0.1:8787/v1/questions \
  -H 'x-user-id: student-demo-a' -H 'x-user-role: student' \
  -H 'Content-Type: application/json' \
  -d '{ "courseId": "SE101", "questionText": "What is REST and which HTTP methods are commonly
used?" }'
```

```
HTTP 201
{ "data": {
    "question": { "id": "<question-id>", "status": "answered", "...": "..." },
    "answer": { "id": "<answer-id>", "mode": "fallback", "noSourceFound": false,
      "content": "Based on the approved course materials, the following source(s) appear
relevant..." },
    "citations": [ { "id": "<citation-id>", "sourceId": "<source-id>",
      "excerpt": "REST is an architectural style for networked applications...", "score": 16 }
],
    "note": "This is a deterministic keyword search over approved course materials, not an AI-
generated explanation."
} }
```

No-source-found question

```
curl -s -X POST http://127.0.0.1:8787/v1/questions \
  -H 'x-user-id: student-demo-a' -H 'x-user-role: student' \
  -H 'Content-Type: application/json' \
  -d '{ "courseId": "SE101", "questionText": "Where can I check my exam schedule?" }'
```

```
HTTP 201
{ "data": {
    "question": { "status": "no_source_found", "...": "..." },
    "answer": { "content": "I could not find an answer in the approved materials for this
course. Please check with your instructor.",
      "mode": "fallback", "noSourceFound": true },
    "citations": [],
    "note": "This is a deterministic keyword search over approved course materials, not an AI-
generated explanation."
} }
```

Question authorization

Caller	Result
student-demo-a (the asking student)	HTTP 200
student-demo-b (a different student)	HTTP 403 FORBIDDEN
instructor-demo	HTTP 200

Feedback

```
curl -s -X POST http://127.0.0.1:8787/v1/answers/feedback \
  -H 'x-user-id: student-demo-a' -H 'x-user-role: student' \
  -H 'Content-Type: application/json' \
  -d '{ "answerId": "<answer-id>", "rating": 5, "comment": "The cited source was helpful." }'
```

```
HTTP 201
{ "data": { "id": "<feedback-id>", "answerId": "<answer-id>", "userId": "student-demo-a",
  "rating": 5, "comment": "The cited source was helpful.", "createdAt": "2026-09-
20T16:37:15.019Z" } }
```

9. Data Quality, Security, and Integrity

- **Request validation.** Dependency-free helpers reject malformed JSON, non-object bodies, empty required strings, and wrong types before any database access (400 VALIDATION_ERROR).
- **Temporary role-based authorization.** x-user-id / x-user-role middleware rejects missing/invalid identity (401) and a route-level role guard rejects a disallowed role (403).
- **Student question ownership.** A student can only read their own question; an instructor can read any question in a course — enforced server-side.

- **Instructor-only source management.** Create/update/archive of knowledge sources is restricted to the instructor role.
- **Parameterized SQL only.** All queries use D1's bound-parameter API; dynamic PATCH updates build from a fixed column whitelist, never arbitrary client keys.
- **Foreign keys and CHECK constraints.** Relationships and finite value sets are enforced by the database itself — verified directly (§10).
- **Correlation IDs.** Every response carries x-correlation-id; every error body includes the same value.
- **Archived-source behavior.** Excluded from future retrieval; existing citations remain intact.
- **Approved-source-only fallback retrieval.** Only status = approved sources in the question's own course are ever searched or cited.
- **No source means no invented answer.** A fixed, honest message is returned instead of a guess.

10. Testing and Verification

- **Type check:** `npm run typecheck` → pass, no errors.
- **Automated tests:** `npm test` → **47 / 47 passing** across 6 test files.
- **Local migration:** applied cleanly to a fresh local D1 — 15 statements, all six tables and eight indexes present; both column defaults confirmed by insert-then-read verification.
- **Local API smoke test:** all seven business endpoints plus health exercised end-to-end against a real local D1 database — source CRUD/archive/idempotency, draft exclusion, matching and non-matching question flows, question-ownership authorization, and feedback creation.
- **Known limitation — plain Node Vitest vs. live D1:** the automated suite runs in a plain Node environment (no `@cloudflare/vitest-pool-workers`), so database-backed success paths are verified manually (above) rather than by an automated integration test. The 47 automated tests cover pure logic and every route's pre-database-access path.
- **Known limitation — simple keyword matching:** the fallback matcher does case-insensitive substring matching on tokens of length ≥ 2, so a very common short token can occasionally match incidentally against unrelated text containing that substring. This is a known characteristic of the current simple-by-design matching approach.
- **Remote migration and schema verification:** performed, with explicit team approval. `wrangler d1 migrations apply learning-assistant-db --remote` reported the migration was already applied (no changes made); a direct `sqlite_master` query against the remote database confirmed all 6 tables and 8 indexes are present and match the local schema. Full evidence in `docs/assignment-3-evidence/remote-migration-verification.txt`.
- **Remote API testing:** not performed. The Worker itself has not been deployed (`wrangler deploy`) or exercised against the remote database — only the schema was verified remotely.

11. Conclusion

The Learning Assistant schema — six tables, eight indexes, foreign keys, and CHECK constraints in a single migration file — directly supports the product's core promise: an answer is either grounded in a specific, approved,

citable source, or the system says so honestly. Cloudflare D1 was a good fit because the domain is small and relational, the Worker binding requires no extra infrastructure, and migrations are simple to apply both locally and remotely. The seven /v1 REST endpoints provide full CRUD (create/read/update/archive) over knowledge sources, plus the question → fallback-answer → citation → feedback flow, all demonstrated above against a real local database with representative fictional data. Remote deployment is designed and ready but intentionally gated behind explicit team approval, consistent with treating a production database migration as a deliberate, reviewed action.

Appendix A: Team Contribution

(To be completed by the team — no contributions are claimed here.)

Team member	Main responsibility	Evidence / commit / document section
TODO	TODO	TODO
TODO	TODO	TODO
TODO	TODO	TODO
TODO	TODO	TODO
TODO	TODO	TODO

Appendix B: Local Commands

```
# Type check and tests
npm run typecheck
npm test

# Reset and apply the local D1 migration
rm -rf .wrangler/state/v3/d1
npx wrangler d1 migrations apply learning-assistant-db --local

# Inspect the local schema
npx wrangler d1 execute learning-assistant-db --local --command \
  "SELECT name, type FROM sqlite_master WHERE type IN ('table', 'index') ORDER BY type, name;"

# Run the Worker locally (forces the D1 binding to local)
npx wrangler dev --local

# Clean up local scratch D1 state when done
rm -rf .wrangler/state/v3/d1
```

To export this page to PDF: open it in a browser and use **Print** → **Save as PDF** (or it may already have been rendered to `output/assignment-3-data-query-design.pdf` — see the task report for confirmation).